Findings on Long Range Communications

A Technical Report

Ahsanul Hoque

Computer Science and Engineering Department
University of Chittagong

ahsanul.csecu@gmail.com

June 2, 2025

Contents

1 Introduction to LoRa **2**

2 LoRa Basis: Chirp Waveform **2**

3 Comparison to Other Modulation Schemes **2**

4 LoRa as Spread Spectrum **3**

5 Types of Chirps and Symbol Formation **3**

6 LoRa Terminology and Spreading Factor (SF) **3**

7 Mathematical Description of LoRa Symbols **3**

8 Visualizing LoRa Symbols **4**

9 LoRa Demodulation (Concept) **5**

10 Understanding Correlation in LoRa Demodulation **5**

11 Challenges in Demodulation **7**

12 Synchronization using the Preamble **7**

13 Initial Receiver Architecture **7**

14 The Mathematical Trick Explained **7**

15 Efficient Receiver Implementation **7**

16 Benefits of the Efficient Receiver **7**

17 LoRa Performance Below the Noise Floor **8**

18 Performance and Spreading Factor vs. SNR **8**

19 LoRa Signal Performance Analysis from MATLAB Simulation **8**

20 LoRa and LoRaWAN Simulation **9**

21 Introduction to LoRa and LoRaWAN Simulation **10**

22 LoRa (Physical Layer) **10**

23 LoRaWAN (MAC and Network Layer Protocol) **10**

24 Introduction to Major Simulation Platforms **12**

25 LoRaWAN Simulator Comparison **12**

LoRa Modulation and Simulation Methods

1 Introduction to LoRa

LoRa stands for **Long Range** and is a relatively new communication standard that focuses on low power, long range, and low bit rate communication. It is widely used in applications such as the Internet of Things (IoT) and remote sensors that demand low power consumption and moderate to low bandwidth. LoRa has the unique capability of demodulating symbols even when they are below the noise floor, making it particularly suitable for long-range communication.

2 LoRa Basis: Chirp Waveform

The foundational waveform in LoRa modulation is the **chirp**, a signal with a linear frequency increase over time. In the time domain, this appears as a sinusoid whose frequency increases linearly, and in a frequency vs. time plot, it appears as a straight line.

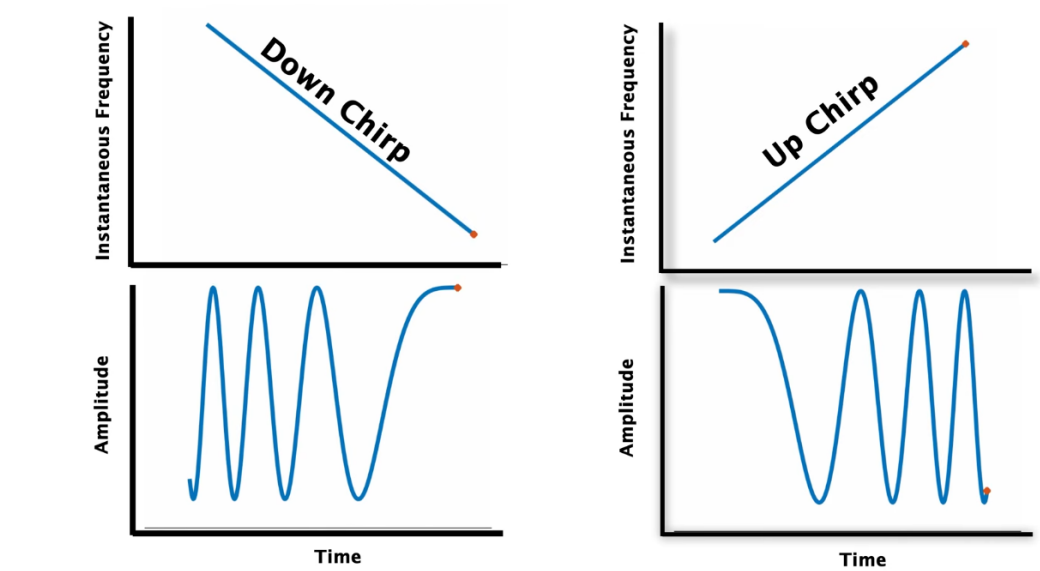


Figure 1: Chirp Spread Spectrum - CSS Spiess, 2021

3 Comparison to Other Modulation Schemes

Traditional schemes like **Phase Shift Keying (PSK)** encode information in the phase of a carrier, and **Frequency Shift Keying (FSK)** uses discrete frequencies to represent different symbols. For instance, FSK with 4 frequencies represents 2 bits per symbol. LoRa differentiates itself by using chirps as the basis for each symbol.

4 LoRa as Spread Spectrum

LoRa is categorized as a **Chirp Spread Spectrum (CSS)** modulation scheme, meaning that each symbol's energy is distributed across a wide bandwidth.

5 Types of Chirps and Symbol Formation

LoRa uses **up chirps** (increasing frequency) and **down chirps** (decreasing frequency). A key question arises: how does LoRa form a large set of distinguishable symbols using just these basic chirps?

6 LoRa Terminology and Spreading Factor (SF)

LoRa is also referred to as **Frequency Shift Chirp Modulation (FSCM)**. The number of possible symbols is 2^{SF}, where SF is the spreading factor (typically between 7 and 12). For example, with $SF = 7$, there are $2^7 = 128$ symbols, allowing transmission of 7 bits per symbol. Each symbol is indexed by an integer s in $[0, 2^{SF} - 1]$.

II. Frequency Shift Chirp Modulation

Let us suppose that the bandwidth of the channel we use for the transmission is B so we transmit a sample every $T = \frac{1}{B}$..

A symbol $s(nT_s)$ is sent at the input of the modulator every $T_s = 2^{SF} \cdot T$. The symbol $s(nT_s)$ is a real number formed using a vector $\mathbf{w}(nT_s)$ of SF binary digits, with SF an integer parameter called, in the context of LoRa, *Spreading Factor* (usually taking values in $\{7, 8, 9, 10, 11, 12\}$) i.e.,

$$s(nT_s) = \sum_{h=0}^{SF-1} \mathbf{w}(nT_s)_h \cdot 2^h. \tag{1}$$

We can see that $s(nT_s)$ takes values in $\{0, 1, 2, \ldots, 2^{SF} - 1\}$.

The transmitted waveform, of duration T_s, for a certain $s(nT_s)$ is then

$$c(nT_s + kT) = \frac{1}{\sqrt{2^{SF}}} e^{j2\pi\left[(s(nT_s)+k) \bmod 2^{SF}\right] kT \frac{B}{2^{SF}}} \tag{2}$$

$$= \frac{1}{\sqrt{2^{SF}}} e^{j2\pi\left[(s(nT_s)+k) \bmod 2^{SF}\right] \frac{k}{2^{SF}}} \tag{3}$$

$$\text{for } k = 0 \ldots 2^{SF} - 1.$$

Figure 2: FREQUENCY SHIFT CHIRP MODULATION(FSCM). from Vangelista, 2017

7 Mathematical Description of LoRa Symbols

Each symbol waveform is a function of:

- Bandwidth B

- Symbol duration $T_s = 2^{SF} * t$
- Sample duration $t = 1/B$
- Symbol $s(nT_s)$
- Sample index k
- Symbol number $s = 0, 1, 2...2^{SF} - 1$
- Modulo operator
- Bandwidth limit 2^{SF}
- Complex Sinusoidal Wave $e^{j\omega t} = \cos\omega t + jsin\omega t$

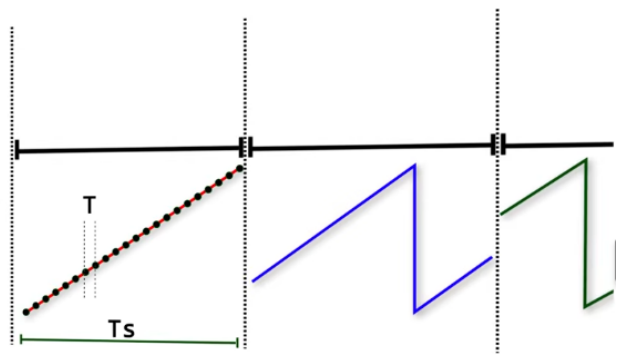


Figure 3: Symbol duration & Sample duration Spiess, 2021

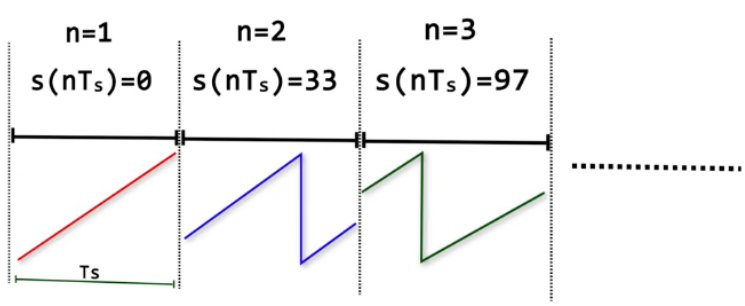


Figure 4: Wave representation of Symbols Spiess, 2021

Symbol number s determines the starting frequency of the chirp. As k increases, the instantaneous frequency increases. Once the frequency exceeds the bandwidth(B), it wraps around using the modulo operator, creating a **discontinuity** (as soon as it hits the bandwidth limit). The location of this discontinuity encodes the symbol. The end frequency of the chirp is the same as the starting frequency.

8 Visualizing LoRa Symbols

By plotting frequency over time, one can visualize the waveform and identify the symbol by locating the discontinuity.

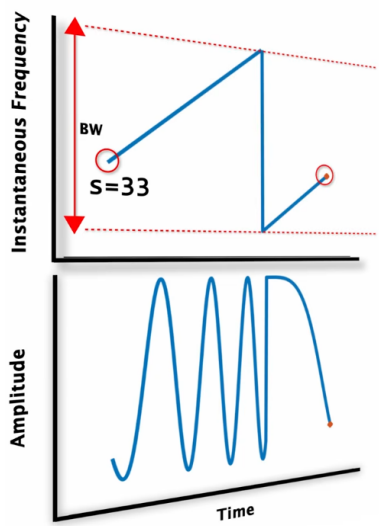


Figure 5: LoRa Transmission Spiess, 2021

9 LoRa Demodulation (Concept)

The receiver determines the transmitted symbol by checking its similarity to all possible symbols for the given SF. For $SF = 7$, this involves comparing against 128 predefined waveforms.

10 Understanding Correlation in LoRa Demodulation

Based on the sources, **correlation** is a fundamental mathematical technique used in LoRa demodulation to determine which symbol was sent. Here's a more detailed explanation:

1. **The Core Concept: Similarity Check**
   The underlying principle of LoRa demodulation is performing a similarity check between the received symbol waveform and all possible symbol waveforms for a specific spreading factor (SF). It is analogous to checking how similar a received color is to a set of known colors to find the closest match.

2. **Correlation as the Similarity Tool**
   This similarity check is performed using **correlation**. Correlation involves mathematically comparing the received signal against each of the potential symbol waveforms.

3. **How Correlation Works (Mathematically)**
   Correlation is calculated by performing a sample-by-sample multiplication of the two waveforms being compared (the received signal and a known symbol waveform), and then summing up the results:

   $$\text{Correlation}(x, y) = \sum_{n=1}^{2^{SF}} a_n \cdot b_n$$

   where a_n and b_n are the n-th samples of the two signals (transmitted and recieved).

4. **Interpreting Correlation Results**
   If the two waveforms are similar, the correlation value will be large. The receiver performs this check for every possible symbol (there are 2^{SF} symbols), and the one with the highest correlation value is determined to be the most likely received symbol.

5. **Alternative Terminology**
   Mathematically, correlation is also referred to as the *inner product*, *dot product*, or, in this context, the *projection* of the received waveform onto the set of possible basis functions.

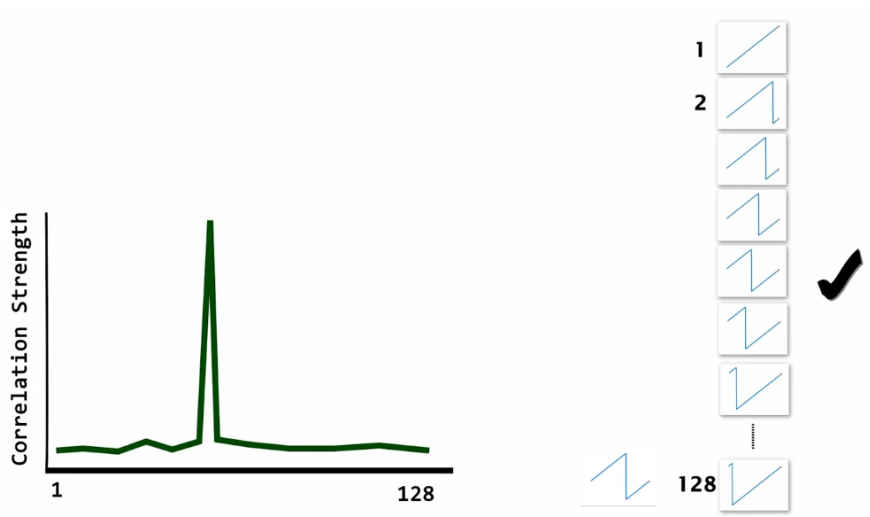


Figure 6: Peak: Highest similarity after matching with all the predetermined values. Spiess, 2021

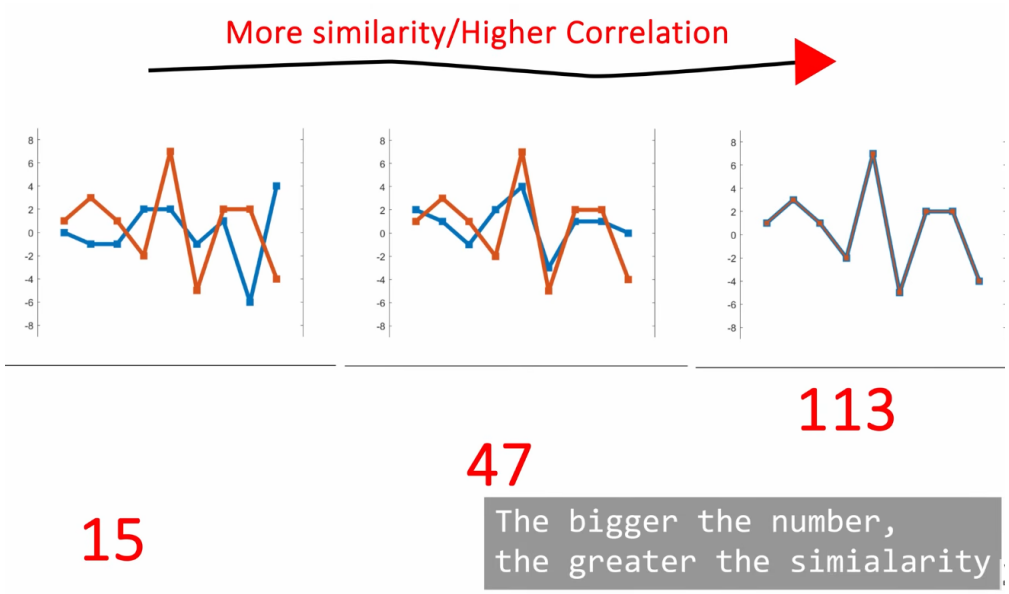


Figure 7: Projection of the basis function of all possibilities Spiess, 2021

6. **Why Correlation is Crucial for LoRa's Performance**
   Correlation enables LoRa to demodulate signals that are below the noise floor. Even when the signal is much weaker than the noise, correlation can identify it by accumulating small similarities across many samples. For instance, with a spreading factor of 12 (SF=12), LoRa can achieve successful communication even at $-20\,\mathrm{dB}$ SNR, thanks to this process.

11 Challenges in Demodulation

Naive correlation with all 2^{SF} symbols is computationally expensive and inefficient, especially for low-power devices. Moreover, communication systems require **synchronization** to align symbol windows correctly.

12 Synchronization using the Preamble

LoRa achieves synchronization by sending a repeated base chirp as a **preamble**. The receiver initially misaligns its detection window, but the repeated symbol allows it to detect a frame and align its window. For instance, if symbol 28 is detected repeatedly, it suggests a misalignment of 28 samples.

13 Initial Receiver Architecture

A basic receiver consists of:

- A **synchronizer** for frame detection
- A **correlator** to compare against all possible symbols

However, this correlator demands 2^{SF} multiplications and additions per symbol, making it inefficient.

14 The Mathematical Trick Explained

LoRa simplifies this using a trick in the **complex baseband**. Correlation with chirps is done by multiplying the received signal by the complex conjugate of the base symbols. Since the conjugate of an up chirp is a down chirp, the multiplication decomposes into:

- A base down chirp (k^2 term), independent of s
- A pure sinusoidal term dependent on s

15 Efficient Receiver Implementation

Using the decomposition, the receiver performs:

1. **De-chirping**: Multiply the signal by the base down chirp
2. **FFT**: Apply Fast Fourier Transform to identify frequency peaks corresponding to s

Note: For more mathematical analysis, check out the video Spiess, 2021 and the paper Vangelista, 2017.

16 Benefits of the Efficient Receiver

This method removes the need for exhaustive correlation and instead uses a simple multiplication followed by an FFT, dramatically reducing computational and power requirements.

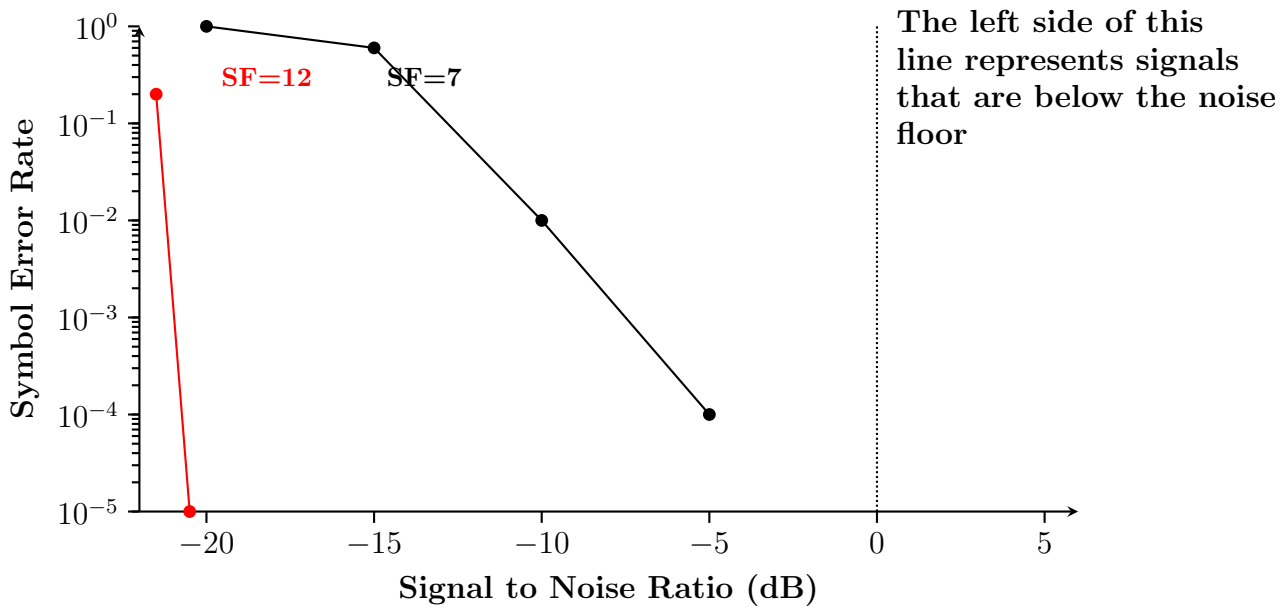
17 LoRa Performance Below the Noise Floor

LoRa can demodulate symbols even when the signal is **below the noise floor** due to the power of correlation. This is a key advantage for long-range, low-power applications.

18 Performance and Spreading Factor vs. SNR

A plot of Symbol Error Rate (SER) vs. Signal-to-Noise Ratio (SNR) shows that:

- For $SF = 7$, communication is viable down to -10 dB SNR.
- For $SF = 12$, communication is possible even at -20 dB SNR.



Higher spreading factors lead to more samples and better correlation results, improving performance in noisy conditions.

19 LoRa Signal Performance Analysis from MATLAB Simulation

Based on the video Spiess, 2021, the MATLAB simulation of LoRa signal performance demonstrates the following key points:

- The simulation visually illustrates the structure of a LoRa signal using a spectrogram, clearly showing the **preamble**, **Start Frame Delimiter (SFD)**, and **data payload**. The SFD itself includes both synchronization components and a down chirp.
- It highlights LoRa's remarkable capability to operate effectively at **low Signal-to-Noise Ratios (SNRs)**, even **below the noise floor**. Specifically, the simulation shows successful communication at:
  - -10 dB SNR for **SF=7**, and
  - -20 dB or even -23 dB SNR for **SF=12**.
- The simulation demonstrates that **increasing the Spreading Factor (SF)** improves performance under low SNR conditions. A higher SF implies more samples per symbol, which results in greater accumulated energy during the correlation process (evidenced by stronger FFT peaks). This accumulation enables the receiver to detect the signal even when it's deeply buried in noise.

- It showcases the computational efficiency of the **de-chirping and FFT-based detection** method compared to brute-force correlation, which would otherwise be computationally intensive and less practical.

20 LoRa and LoRaWAN Simulation

- LoRa and LoRaWAN enable low-power, long-range wireless communication for IoT.
- Simulation helps in optimizing and understanding network behavior before physical deployment.

LoRa Physical Layer (PHY)

- LoRa uses Chirp Spread Spectrum (CSS), achieving long-range communication with low power.
- Operates in sub-GHz ISM bands (868 MHz in EU, 915 MHz in NA, 433 MHz in Asia).
- Key parameters:
  - **Spreading Factor (SF)**: Higher SF = longer range, lower data rate, more energy.
  - **Bandwidth (BW)**: 125/250/500 kHz; wider BW = higher data rate, lower sensitivity.
  - **Coding Rate (CR)**: Forward Error Correction (FEC) ratios; more redundancy improves robustness.
  - **Transmission Power**: Must comply with regional limits.
- Trade-offs among these parameters affect energy, throughput, and range.

LoRaWAN MAC Layer and Network Architecture

- Defines protocol and system architecture.
- Topology: Star-of-stars. Devices → Gateways → Network Server → Application Server.
- Device Classes:
  - **Class A**: Most power-efficient. Uplink opens two short downlink windows.
  - **Class B**: Adds scheduled receive windows using beacon synchronization.
  - **Class C**: Nearly always listening; lowest latency, highest power.
- Adaptive Data Rate (ADR): Dynamically adjusts SF, BW, and TxPower to optimize network and battery.

Simulation Model Fidelity

- LoRa's complexity (CSS, quasi-orthogonal SFs, variable wireless channels) makes analytical modeling hard.
- Simulators use mathematical and empirical models for:
  - Radio propagation: Free Space, Log-distance, Two-Ray, Okumura-Hata.
  - Interference: Threshold-based, SINR-based, collision matrices.
  - Energy consumption modeling.
- Users must understand model assumptions and limitations to ensure credible results.

21 Introduction to LoRa and LoRaWAN Simulation

- LoRa and LoRaWAN enable low-power, long-range wireless communication for IoT.
- Simulation helps in optimizing and understanding network behavior before physical deployment.

22 LoRa (Physical Layer)

LoRa is a physical layer (PHY) modulation technology responsible for encoding data into radio waves using Chirp Spread Spectrum (CSS). It defines how bits are sent and received wirelessly over long distances with low power consumption.

LoRa does not inherently support specific higher-level communication protocols. Instead, various communication protocols can be built on top of the LoRa physical layer. Think of LoRa as the *carrier wave* technology.

While it's technically possible to create proprietary point-to-point or mesh protocols using LoRa as the PHY layer, such use is less common in wide-area IoT applications compared to standardized LoRaWAN.

23 LoRaWAN (MAC and Network Layer Protocol)

LoRaWAN is the most common protocol stack built on top of the LoRa physical layer. It defines both the Media Access Control (MAC) layer and the network architecture.

LoRaWAN MAC Protocol

The MAC layer manages communication between end-devices and gateways. Key components include:

- **Device Classes (A, B, C):** Define how devices manage uplink and downlink communication, balancing power consumption and latency.
- **Frame Format:** Specifies the structure of LoRaWAN packets, including headers, payload, and message integrity codes (MIC).
- **Addressing:** Unique identifiers such as DevEUI and DevAddr.
- **Join Procedure:** Secure mechanisms for network access:
  - OTAA – Over-The-Air Activation
  - ABP – Activation By Personalization
- **MAC Commands:** Used for network management, including:
  - Adaptive Data Rate (ADR)
  - Duty cycle enforcement
  - Channel plan management
  - Device status requests
- **Security:** End-to-end encryption using AES-128 for both:
  - Network session key (NwkSKey)
  - Application session key (AppSKey)

Application Layer Protocols over LoRaWAN

LoRaWAN is application-layer agnostic. It transparently transports application payloads. Payload formats are defined by the application and may include:

- **Custom Binary Formats:** Efficient packing of sensor data.
- **Cayenne LPP (Low Power Payload):** Standardized compact format for multiple sensor types.
- **CBOR (Concise Binary Object Representation):** A more compact alternative to JSON.
- **Proprietary Formats:** Specific to application or manufacturer.

Although technically possible, IP (Internet Protocol) is rarely used directly over LoRaWAN due to:

- Small payload sizes
- High overhead of IP headers
- Low data rates

LoRaWAN is optimized for small, infrequent data transmissions.

Gateway-to-Network Server Protocols (Backhaul)

Gateways connect to the Network Server (NS) over standard IP networks (Ethernet, Wi-Fi, or Cellular). Common backhaul protocols include:

- **Semtech UDP Packet Forwarder Protocol (GWMP):**
  - Simple and widely used
  - Lacks reliability and security (e.g., no TLS)
- **LoRaWAN Backend Interfaces:**
  - Defined by the LoRa Alliance
  - Enable communication between Network Server, Application Server, and Join Server
  - Use HTTPS/REST APIs or MQTT over TCP/IP
- **MQTT (Message Queuing Telemetry Transport):**
  - Lightweight, publish/subscribe model
  - Supports TLS security
  - Increasingly used in modern systems
- **CoAP (Constrained Application Protocol):**
  - Runs over UDP
  - Suitable for constrained environments

24 Introduction to Major Simulation Platforms

- **General-Purpose Network Simulators**:
  - **ns-3**: Open-source, C++/Python, lorawan module by Signet Lab enables detailed LoRaWAN simulations.
  - **OMNeT++**: Modular, graphical NED language, FLoRa framework integrates with INET for LoRaWAN modeling.
- **Language-Specific Simulators (Python/SimPy)**:
  - **LoRaSim** and **LoRaSimPlus**: Python-based, prioritize rapid prototyping, scalability, and collision analysis.
- **LoRaWAN Network Server (LNS) Integrated Simulators**:
  - **ChirpStack Simulator**: Integrates with ChirpStack LNS to simulate many devices and gateways for load testing.
- **Other Simulators**:
  - **CupCarbon**: Java-based, smart city IoT simulations.
  - **PhySimulator**: MATLAB-based, focused on link-level PHY modeling.
- **Utility Tools**:
  - **Semtech LoRa Calculator**: Quick estimations for time-on-air, link budget; not a network simulator.
- **Platform Selection Criteria**:
  - Depends on objectives, user background, and simulation scope.
  - Researchers often prefer ns-3 or OMNeT++ for deep customization.
  - Python tools offer ease and agility.
  - ChirpStack tools suit LNS developers focused on real-world interaction.

25 LoRaWAN Simulator Comparison

Simulator Name: ns-3/LoRaWAN module

- **Base Platform / Language:** ns-3 / C++, Python
- **Key LoRaWAN Features Supported:** Class A, ADR, Confirmed/Unconfirmed Msgs, Multi-GW, Network Server Model
- **LoRa PHY Parameter Configuration:** DataRate attribute (maps to SF/BW), TxPower, CR (via PHY/MAC helpers)
- **Energy Modeling Support:** Yes, detailed (TX, RX, Sleep states), LoraRadioEnergyModel
- **Interference Modeling Support:** Yes (Co-SF, Inter-SF via collision matrix, e.g., GOURSAUD, Capture Effect, Propagation Models)
- **Scalability (Nodes/Gateways):** High (thousands of nodes)
- **Licensing:** GNU GPLv2
- **GUI Availability:** Via NetAnim (for C++), PyViz (for Python)

- **Documentation / Tutorial Quality & Availability:** Good (Official ns-3 docs, module docs, examples)
- **Community Support:** Very Good (Mailing lists, GitHub, forums)
- **Primary Use Cases:** Detailed PHY/MAC research, protocol development, scalability, interference studies, academic research

Simulator Name: LoRaSim

- **Base Platform / Language:** Python
- **Key LoRaWAN Features Supported:** Class A, ADR, Confirmed/Unconfirmed Msgs, Single-GW
- **LoRa PHY Parameter Configuration:** SF, BW, CR, TxPower
- **Energy Modeling Support:** No (Energy stats estimated externally)
- **Interference Modeling Support:** Yes (Co-SF, Inter-SF, Capture Effect, basic collision modeling)
- **Scalability (Nodes/Gateways):** Medium (hundreds of nodes)
- **Licensing:** GNU GPL
- **GUI Availability:** No
- **Documentation / Tutorial Quality & Availability:** Moderate (some examples, paper-based guidance)
- **Community Support:** Limited
- **Primary Use Cases:** Basic performance analysis, interference, scalability studies

Simulator Name: OMNeT++ / FLoRa

- **Base Platform / Language:** OMNeT++, INET / C++
- **Key LoRaWAN Features Supported:** Class A, Confirmed Msgs, Multi-GW
- **LoRa PHY Parameter Configuration:** SF, BW, TxPower
- **Energy Modeling Support:** Yes (via INET framework)
- **Interference Modeling Support:** Yes (via radio medium & propagation model abstraction)
- **Scalability (Nodes/Gateways):** High
- **Licensing:** Academic Public License (OMNeT++) / Custom for FLoRa
- **GUI Availability:** Yes (via OMNeT++ IDE)
- **Documentation / Tutorial Quality & Availability:** Good (via INET & OMNeT++, moderate for FLoRa)
- **Community Support:** Moderate to Good (via OMNeT++ forums, INET)
- **Primary Use Cases:** Protocol design and validation, medium-scale simulations

Simulator Name: CupCarbon

- **Base Platform / Language:** Java
- **Key LoRaWAN Features Supported:** Basic LoRa physical layer simulation, limited LoRaWAN support
- **LoRa PHY Parameter Configuration:** SF, BW, TxPower, basic CR
- **Energy Modeling Support:** Yes (Sensor power models included)
- **Interference Modeling Support:** Basic
- **Scalability (Nodes/Gateways):** Medium
- **Licensing:** Free for academic use
- **GUI Availability:** Yes (Integrated GUI)
- **Documentation / Tutorial Quality & Availability:** Moderate (academic papers and online guides)
- **Community Support:** Limited
- **Primary Use Cases:** Education, basic network design, geographic simulations

Simulator Name: NS-3 LENA + LoRa extension

- **Base Platform / Language:** ns-3 / C++
- **Key LoRaWAN Features Supported:** LoRa PHY + LTE coexistence simulation
- **LoRa PHY Parameter Configuration:** Yes, supports multiple PHY configurations
- **Energy Modeling Support:** Yes
- **Interference Modeling Support:** Yes (LTE vs LoRa interference simulation)
- **Scalability (Nodes/Gateways):** High
- **Licensing:** GNU GPL
- **GUI Availability:** Limited (via NetAnim or trace file visualization)
- **Documentation / Tutorial Quality & Availability:** Moderate
- **Community Support:** Moderate
- **Primary Use Cases:** LTE/LoRa coexistence studies

Simulator Name: PhySimulator

- **Base Platform / Language:** MATLAB
- **Key LoRaWAN Features Supported:** Physical layer focused simulation
- **LoRa PHY Parameter Configuration:** Full PHY parameter configurability
- **Energy Modeling Support:** No
- **Interference Modeling Support:** Yes (PHY signal analysis, custom interference)
- **Scalability (Nodes/Gateways):** Low to Medium

- **Licensing:** MATLAB proprietary
- **GUI Availability:** MATLAB GUI based
- **Documentation / Tutorial Quality & Availability:** Moderate (papers + code)
- **Community Support:** Limited
- **Primary Use Cases:** LoRa PHY layer research, teaching

References

DroneBot Workshop. (2023). Lora - long-range radio for iot — arduino, esp32, rpi pico [YouTube Video].

NetworkSimulationTools. (2022). Simulation and analysis of intelligent collision avoidance [Accessed: 2025-06-02].

Pothos. (2013). Lora modem simulation in pothos [Accessed: 2025-06-02].

SimpleSoft, Inc. (2013). Simpleiotsimulator - iot / lora simulator - create 5,000 sensors in minutes [Accessed: 2025-06-02].

SimpleSoft, Inc. (2021). Using simpleiotsimulator for testing lora chirpstack network server [Accessed: 2025-06-02].

Spiess, A. (2021). How lora modulation really works – long range communication using chirps [YouTube Video]. https://youtu.be/jHWepP1ZWTk?si=d7CIRY7yW9WXZamP

TiiLaz. (2023). Lora based wireless sensor network in iot simulation [Accessed: 2025-06-02].

Vangelista, L. (2017). Analysis and implementation of a lora modem based on software defined radio. *IEEE Signal Processing Letters*, *24*(12), 1818–1821. https://doi.org/10.1109/LSP.2017.2762960